



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

Практична робота №1

Моделі математичної економіки та прийняття рішень в інформаційних системах

Тема: Методи прийняття рішень на основі бінарних відношень та попарних
порівнянь

Варіант 6

Виконав:

студент групи ІІІ-51мн

Панченко С. В.

Перевірив:

Поперешняк С. В.

ЗМІСТ

1	Мета	3
2	Блок 1. Постановка основних задач прийняття рішень	4
2.1	Задача вибору оптимального шляху за критерієм мінімізації часу	4
2.2	Задача вибору параметрів логічної системи за заданими критеріями	4
2.3	Задача прийняття рішення за наявності двох альтернатив	4
2.4	Задача прийняття рішення в умовах невизначеності типу «контрагент» .	4
2.5	Задача прийняття рішення за відсутності числових даних	4
2.6	Принципи прийняття рішень у погано формалізованих задачах	5
3	Блок 2. Прийняття рішень на основі бінарних відношень (варіант б)	6
3.1	Вихідні дані	6
3.2	Перевірка властивостей відношення	6
3.3	Обернене та доповняльне відношення	7
3.4	Найбільший, найменший, максимальні та мінімальні елементи	7
4	Блок 3. Попарні порівняння в задачах прийняття рішень	8
4.1	Постановка	8
4.2	Матриця попарних порівнянь	8
4.3	Ранжирування методом рядкових сум	8
4.4	Перевірка узгодженості (транзитивності)	9
5	Висновки	10
6	Лістинг коду	11
6.1	binary_relations.py	11

1 МЕТА

Ознайомитися з постановками основних задач прийняття рішень; ознайомитися з властивостями бінарних відношень та операцій над ними, набути навичок щодо прийняття рішень на основі заданих відношень; ознайомитися з методикою побудови матриці попарних порівнянь (МПП) в задачах прийняття рішення.

Усі розрахунки виконано мовою Python (бібліотека NumPy для матричних операцій) замість Excel, а звіт підготовлено засобами Typst замість MS Word.

2 БЛОК 1. ПОСТАНОВКА ОСНОВНИХ ЗАДАЧ ПРИЙНЯТТЯ РІШЕНЬ

2.1 Задача вибору оптимального шляху за критерієм мінімізації часу

Об'єкту потрібно дістатися з пункту А до пункту В, долаючи ділянки з різною швидкістю пересування (наприклад, асфальтову й піщану). Альтернативи параметризуються точкою переходу X між ділянками, а результат кожної альтернативи — часом пересування. Оскільки кожній альтернативі відповідає однозначний числовий результат, це — задача прийняття рішення в умовах визначеності.

2.2 Задача вибору параметрів логічної системи за заданими критеріями

Проектувальник обирає параметри схеми (наприклад, опори резисторів), яка характеризується двома показниками — потужністю споживання f_1 і часом затримки сигналу f_2 . Кожному набору параметрів відповідає точка на площині (f_1, f_2) . Задача ускладнена наявністю двох критеріїв одночасно: покращення одного може погіршувати інший, тому виникає не обчислювальна, а концептуальна складність — що саме вважати оптимальним результатом.

2.3 Задача прийняття рішення за наявності двох альтернатив

Приклад — пасажир трамвая вирішує, купувати квиток чи ні, а результат залежить від того, чи з'явиться контролер. Тут особа, яка приймає рішення (ОПР), має дві альтернативи, а середовище — два стани; оцінки результатів задаються таблицею грошових втрат. Це задача прийняття рішення в умовах невизначеності (природної, оскільки стан середовища не залежить від волі опонента).

2.4 Задача прийняття рішення в умовах невизначеності типу «контрагент»

Класичний приклад — «дилема ув'язненого»: двоє підозрюваних незалежно обирають стратегію (визнати провину чи заперечувати), а результат для кожного залежить від стратегії іншого. На відміну від «природної» невизначеності, тут невизначеність зумовлена не середовищем, а поведінкою розумного контрагента, і ефективність рішення залежить від взаємної інформованості сторін про наміри одна одної.

2.5 Задача прийняття рішення за відсутності числових даних

ОПР може лише впорядкувати результати попарно (без чисельних оцінок). Наприклад, вибір місця роботи молодим науковцем із трьох альтернатив, коли

попарні переваги можуть виявитися нетранзитивними ($x_1 > x_2$, $x_2 > x_3$, але $x_3 > x_1$) — найкращої альтернативи в такому разі обрати неможливо без додаткових принципів.

2.6 Принципи прийняття рішень у погано формалізованих задачах

Задачі на кшталт медичної діагностики не мають адекватного традиційного математичного опису й розв'язуються за допомогою експертних систем на основі бази знань та продукційних правил виду «ЯКЩО (умова), ТО (дія)». Головну роль тут відіграють методи формування бази знань для вузької предметної галузі та процедури логічного висновку на основі вихідних фактів.

3 БЛОК 2. ПРИЙНЯТТЯ РІШЕНЬ НА ОСНОВІ БІНАРНИХ ВІДНОШЕНЬ (ВАРІАНТ 6)

3.1 Вихідні дані

Для варіанта 6 (Додаток А) задано відношення R на множині $X = \{x_1, \dots, x_5\}$ матрицею:

$$R = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 \end{pmatrix}.$$

3.2 Перевірка властивостей відношення

Властивості перевірено безпосереднім аналізом матриці засобами NumPy:

- рефлексивність: на головній діагоналі R стоять лише одиниці $\rightarrow$ рефлексивне: так
- антирефлексивність: ні (діагональ не нульова);
- симетричність: $R \neq R^T$ (наприклад, $a_{2,4} = 1$, але $a_{4,2} = 0$) $\rightarrow$ симетричне: ні
- асиметричність: ні (рефлексивність уже виключає асиметричність, оскільки $a_{i,i} \wedge a_{i,i} = 1 \neq 0$);
- антисиметричність: для всіх позадіагональних пар $a_{i,j} \wedge a_{j,i} = 0 \rightarrow$ антисиметричне: так
- транзитивність: обчислено R^2 (булеве множення матриці на себе) і перевірено умову $R^2 \leq R$. Оскільки, наприклад, $R_{2,1}^2 = 1$, а $R_{2,1} = 0$, умова порушується $\rightarrow$ транзитивне: ні.

$$R^2 = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 \end{pmatrix}.$$

3.3 Обернене та доповняльне відношення

$$R^{-1} = R^T = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix};$$

$$\overline{R} = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix}.$$

3.4 Найбільший, найменший, максимальні та мінімальні елементи

Найбільший елемент — той, чий рядок у R складається лише з одиниць. За результатами аналізу найбільші елементи: x_3 (x^*Rx для будь-якого x). Найменші елементи (стовпці з усіма одиницями): немає.

Для визначення максимальних і мінімальних елементів побудовано строге відношення $R^S = R \wedge \neg R^T$ (елемент $a_{i,j}^S = 1$, якщо x_i строго краще за x_j):

$$R^S = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix}.$$

Максимальний елемент визначається порожнім стовпцем у R^S (ніхто не домінує над ним): максимальні елементи — x_3 . Мінімальний елемент визначається порожнім рядком у R^S (він нікого строго не домінує): мінімальні елементи — x_1 .

Висновок по Блоку 2. Відношення варіанта 6 є рефлексивним, антисиметричним, несиметричним і нетранзитивним. Воно має найбільший (і водночас максимальний) елемент x_3 , найменшого елемента не існує, а мінімальним є елемент x_1 .

4 БЛОК 3. ПОПАРНІ ПОРІВНЯННЯ В ЗАДАЧАХ ПРИЙНЯТТЯ РІШЕНЬ

4.1 Постановка

Згенеровано множину із 6 навчальних дисциплін магістерської програми та порівняно їх між собою за суб'єктивною значущістю особисто для себе методом попарних порівнянь:

Таблиця 4.1 Перелік дисциплін для попарного порівняння

Позначення	Дисципліна
a1	Моделі математичної економіки
a2	Технології великих даних
a3	Управління ІТ-проєктами
a4	Розподілені системи
a5	Машинне навчання
a6	Кібербезпека

4.2 Матриця попарних порівнянь

Заповнено верхню трикутну частину МПП (елемент $a_{ij} = 1$, якщо дисципліна i значущіша за j ; -1 — навпаки; 0 — рівнозначні), нижню віддзеркалено. Додано стовпчик рядкових сум:

Таблиця 4.2 Матриця попарних порівнянь (МПП) дисциплін

Дисц.	a1	a2	a3	a4	a5	a6	Σ
a1	0	-1	1	-1	-1	1	-1
a2	1	0	1	1	1	1	5
a3	-1	-1	0	-1	-1	1	-3
a4	1	-1	1	0	-1	1	1
a5	1	-1	1	1	0	1	3
a6	-1	-1	-1	-1	-1	0	-5

4.3 Ранжирування методом рядкових сум

За методом рядкових сум об'єкту з максимальною сумою ω_j відводиться перше місце в ранжируванні:

$a2 (\omega = 5) > a5 (\omega = 3) > a4 (\omega = 1) > a1 (\omega = -1) > a3 (\omega = -3) > a6 (\omega = -5)$

Отже, найбільш значущою суб'єктивно виявилась дисципліна “Технології великих даних”, а найменш значущою — “Кібербезпека”.

4.4 Перевірка узгодженості (транзитивності)

Оскільки МПП заповнюється дзеркально-симетрично, достатньо перевірити всі трійки об'єктів, для яких обидва порівняння в ланцюжку спрямовані в один бік (обидва «>» або обидва «<»); лише в такому разі транзитивність взагалі можна перевірити. З $C_6^3 = 20$ можливих трійок такими виявились 6:

Таблиця 4.3 Перевірка транзитивності на всіх перевірюваних трійках

Трійка	Умова	Очікується / факт	Оцінка
a1,a3,a6	a1>a3, a3>a6	a1>a6, факт a1>a6	+
a1,a4,a5	a1<a4, a4<a5	a1<a5, факт a1<a5	+
a2,a3,a6	a2>a3, a3>a6	a2>a6, факт a2>a6	+
a2,a4,a6	a2>a4, a4>a6	a2>a6, факт a2>a6	+
a2,a5,a6	a2>a5, a5>a6	a2>a6, факт a2>a6	+
a3,a4,a5	a3<a4, a4<a5	a3<a5, факт a3<a5	+

Усі 6 перевірюваних трійок задовольняють умову транзитивності — порушень не виявлено.

Висновок по Блоку 3. На відміну від ілюстративного прикладу методичних вказівок (де 7 із 9 перевірюваних трійок виявили порушення транзитивності), побудована МПП виявилась повністю узгодженою: жодних циклічних тріад не знайдено. Це означає, що особисте ранжирування дисциплін за значущістю є логічно послідовним і не містить суперечностей у попарних судженнях.

5 ВИСНОВКИ

У ході практичної роботи розглянуто основні типи задач прийняття рішень (в умовах визначеності, невизначеності природного та контрагентного типу, за відсутності числових даних, погано формалізовані задачі) та реалізовано програмно (мовою Python, бібліотека NumPy) два методи прийняття рішень.

Аналіз бінарного відношення варіанта 6 показав, що воно є рефлексивним і антисиметричним, але не симетричним і не транзитивним; знайдено єдиний найбільший (максимальний) елемент x_3 та мінімальний елемент x_1 , а найменшого елемента в цьому відношенні не існує.

Метод попарних порівнянь застосовано до самостійно обраної множини з 6 навчальних дисциплін: побудовано МПП, виконано ранжирування методом рядкових сум і перевірено узгодженість оцінок за всіма перевірюваними трійками об'єктів. На відміну від прикладу з методичних вказівок, отримана МПП виявилась повністю транзитивною (узгодженою), що підтверджує логічну послідовність суб'єктивних суджень під час порівняння.

6 ЛІСТИНГ КОДУ

6.1 binary_relations.py

```
"""
```

Практична робота №1 (розділ 2) – Варіант 6

Методи прийняття рішень на основі бінарних відношень та попарних порівнянь

Усі обчислені результати також зберігаються у data.json, який читає безпосередньо Turst-звіт – це виключає розбіжності між текстом звіту та фактичним виводом скрипта.

```
"""
```

```
import itertools
```

```
import json
```

```
import pathlib
```

```
import numpy as np
```

```
np.set_printoptions(linewidth=120)
```

```
SCRIPT_DIR = pathlib.Path(__file__).resolve().parent
```

```
def mat(a):
```

```
    """numpy-масив -> список списків int, придатний для JSON."""
```

```
    return np.asarray(a).astype(int).tolist()
```

```
def analyze_relation(R, label):
```

```
    n = R.shape[0]
```

```
    print(f"=== {label} ===")
```

```
    print(f"R =\n{R}")
```

```
    reflexive = bool(np.all(np.diag(R) == 1))
```

```
    antireflexive = bool(np.all(np.diag(R) == 0))
```

```
    symmetric = bool(np.array_equal(R, R.T))
```

```
    asymmetric = bool(np.all((R & R.T) == 0))
```

```
    off_diag_mask = ~np.eye(n, dtype=bool)
```

```
    antisymmetric = bool(np.all((R & R.T)[off_diag_mask] == 0))
```

```

R2 = ((R @ R) > 0).astype(int)
transitive = bool(np.all(R2 <= R))

print(f"Рефлексивне: {reflexive}")
print(f"Антирефлексивне: {antireflexive}")
print(f"Симетричне: {symmetric}")
print(f"Асиметричне: {asymmetric}")
print(f"Антисиметричне: {antisymmetric}")
print(f" $R^2 = \{R2\}$ ")
print(f"Транзитивне: {transitive}")

R_inv = R.T
R_comp = 1 - R
print(f"Обернене  $R^{-1} = R^T = \{R\_inv\}$ ")
print(f"Доповнення  $\bar{R} = 1 - R = \{R\_comp\}$ ")

greatest = [i for i in range(n) if np.all(R[i, :] == 1)]
least = [i for i in range(n) if np.all(R[:, i] == 1)]
print(f"Найбільші елементи (індекси, з 0): {greatest}")
print(f"Найменші елементи (індекси, з 0): {least}")

R_strict = R * (1 - R.T)
print(f"Строге відношення  $R^S = R \& !R^T = \{R\_strict\}$ ")
maximal = [i for i in range(n) if np.all(R_strict[:, i] == 0)]
minimal = [i for i in range(n) if np.all(R_strict[i, :] == 0)]
print(f"Максимальні елементи (індекси, з 0): {maximal}")
print(f"Мінімальні елементи (індекси, з 0): {minimal}")

return {
    "reflexive": reflexive,
    "antireflexive": antireflexive,
    "symmetric": symmetric,
    "asymmetric": asymmetric,
    "antisymmetric": antisymmetric,
    "transitive": transitive,
    "R_inv": R_inv,
    "R_comp": R_comp,

```

```

        "greatest": greatest,
        "least": least,
        "maximal": maximal,
        "minimal": minimal,
        "R2": R2,
        "R_strict": R_strict,
    }

print("=" * 70)
print("БЛОК 2. Прийняття рішень на основі бінарних відношень (варіант 6)")
print("=" * 70)

# Матриця варіанта 6 (Додаток А)
R6 = np.array([
    [1, 0, 0, 0, 0],
    [0, 1, 0, 1, 0],
    [1, 1, 1, 1, 1],
    [1, 0, 0, 1, 1],
    [0, 1, 0, 0, 1],
])

result6 = analyze_relation(R6, "Варіант 6")

data = {
    "block2": {
        "R": mat(R6),
        "properties": {
            "reflexive": result6["reflexive"],
            "antireflexive": result6["antireflexive"],
            "symmetric": result6["symmetric"],
            "asymmetric": result6["asymmetric"],
            "antisymmetric": result6["antisymmetric"],
            "transitive": result6["transitive"],
        },
        "R2": mat(result6["R2"]),
        "R_inv": mat(result6["R_inv"]),
    }
}

```

```

        "R_comp": mat(result6["R_comp"]),
        "R_strict": mat(result6["R_strict"]),
        "greatest": [i + 1 for i in result6["greatest"]],
        "least": [i + 1 for i in result6["least"]],
        "maximal": [i + 1 for i in result6["maximal"]],
        "minimal": [i + 1 for i in result6["minimal"]],
    }
}

print("\n" + "=" * 70)
print("БЛОК 3. Попарні порівняння в задачах прийняття рішень")
print("=" * 70)

names = [
    "Моделі математичної економіки",
    "Технології великих даних",
    "Управління ІТ-проектами",
    "Розподілені системи",
    "Машинне навчання",
    "Кібербезпека",
]
n = len(names)
codes = [f"a{i+1}" for i in range(n)]

# Верхня трикутна частина МПП: суб'єктивні оцінки значущості дисциплін
# для власного навчального плану (1 - краще, -1 - гірше, 0 - однаково)
pairs = {
    (0, 1): -1, (0, 2): 1, (0, 3): -1, (0, 4): -1, (0, 5): 1,
    (1, 2): 1, (1, 3): 1, (1, 4): 1, (1, 5): 1,
    (2, 3): -1, (2, 4): -1, (2, 5): 1,
    (3, 4): -1, (3, 5): 1,
    (4, 5): 1,
}

M = np.zeros((n, n), dtype=int)
for (i, j), v in pairs.items():
    M[i, j] = v
    M[j, i] = -v

```

```

print("МПП (матриця попарних порівнянь):")
header = "          " + "".join(f"{c:>5}" for c in codes) + "      сума"
print(header)
row_sums = M.sum(axis=1)
for i, code in enumerate(codes):
    print(f"{code:<8}" + "".join(f"{M[i,j]:>5}" for j in range(n)) +
          f"{row_sums[i]:>8}")

order = np.argsort(-row_sums)
print("\nРанжирування (від найбільш значущого до найменш значущого):")
ranking = []
for rank, idx in enumerate(order, start=1):
    print(f"  {rank}. {codes[idx]} – {names[idx]} (сума =
          {row_sums[idx]})")
    ranking.append({
        "rank": rank,
        "code": codes[idx],
        "name": names[idx],
        "sum": int(row_sums[idx]),
    })

def rel_symbol(v):
    return ">" if v == 1 else ("<" if v == -1 else "=")

print("\nПеревірка узгодженості (транзитивності) по всіх трійках
об'єктів:")
print(f"{'№':<4}{'Трійка':<12}{'Умова':<20}{'Висновок':<20}{'Оцінка'}")
total_triads = 0
checkable = 0
violations = 0
triads = []
for idx, (i, j, k) in enumerate(itertools.combinations(range(n), 3),
start=1):
    total_triads += 1
    a_ij, a_jk, a_ik = M[i, j], M[j, k], M[i, k]

```

```

triple = f"{codes[i]},{codes[j]},{codes[k]}"
cond = f"{codes[i]}{rel_symbol(a_ij)}{codes[j]}, {codes[j]}
{rel_symbol(a_jk)}{codes[k]}"
if a_ij == 0 or a_jk == 0 or a_ij != a_jk:
    print(f"{idx:<4}{triple:<12}{cond:<20}{'не
перевіряється':<20}")
    triads.append({
        "triple": triple, "cond": cond,
        "checkable": False, "conclusion": "", "mark": "",
    })
    continue
checkable += 1
expected = a_ij
ok = a_ik == expected
concl = f"{codes[i]}{rel_symbol(expected)}{codes[k]}, факт
{codes[i]}{rel_symbol(a_ik)}{codes[k]}"
mark = "+" if ok else "-"
if not ok:
    violations += 1
print(f"{idx:<4}{triple:<12}{cond:<20}{concl:<20}{mark}")
triads.append({
    "triple": triple, "cond": cond,
    "checkable": True, "conclusion": concl, "mark": mark,
})

print(f"\nВсього трійок: {total_triads}, перевірюваних: {checkable},
порушень транзитивності: {violations}")
print("Матриця узгоджена" if violations == 0 else f"Матриця має
{violations} порушень транзитивності")

data["block3"] = {
    "names": names,
    "codes": codes,
    "M": mat(M),
    "row_sums": [int(x) for x in row_sums],
    "ranking": ranking,
    "triads": triads,
    "total_triads": total_triads,

```

```
    "checkable": checkable,  
    "violations": violations,  
}
```

```
out_path = SCRIPT_DIR / "data.json"  
with open(out_path, "w", encoding="utf-8") as f:  
    json.dump(data, f, ensure_ascii=False, indent=2)  
print(f"\nДані збережено у {out_path}")
```